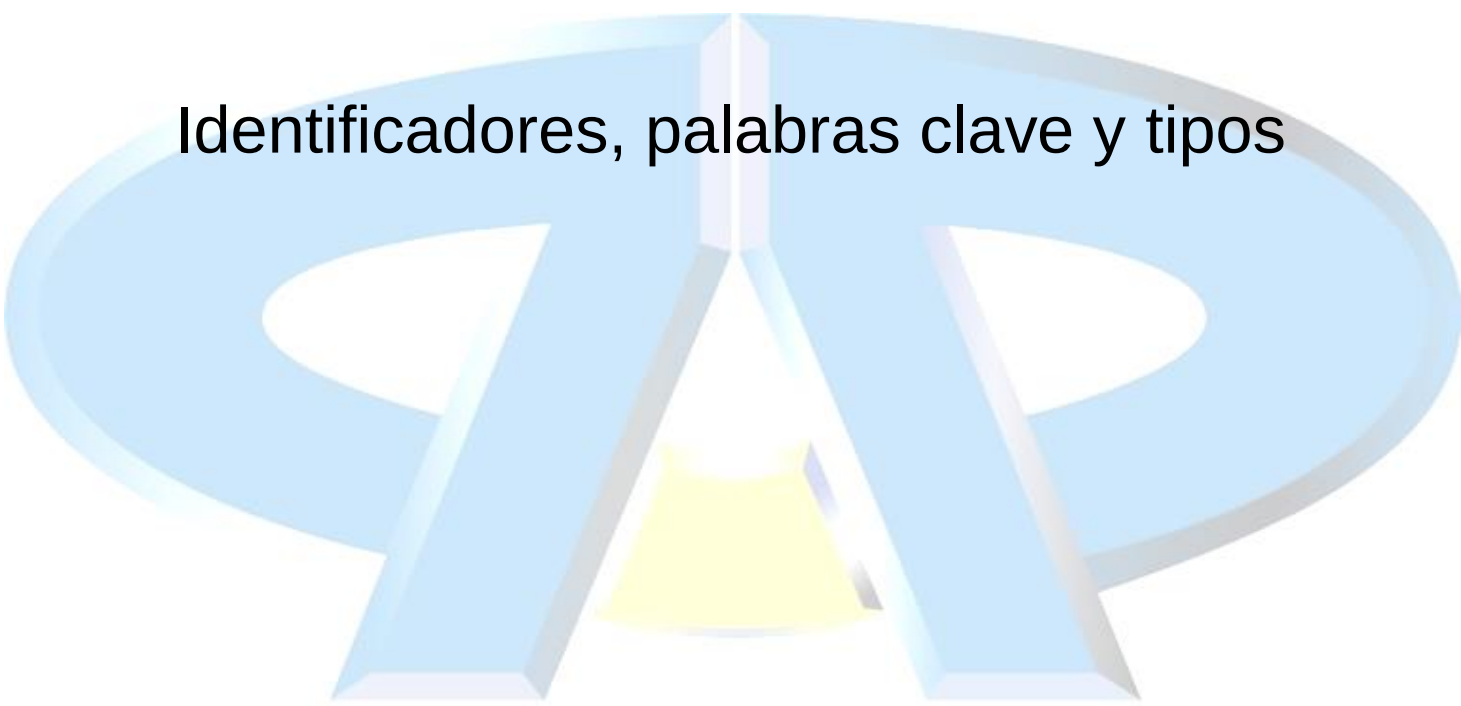
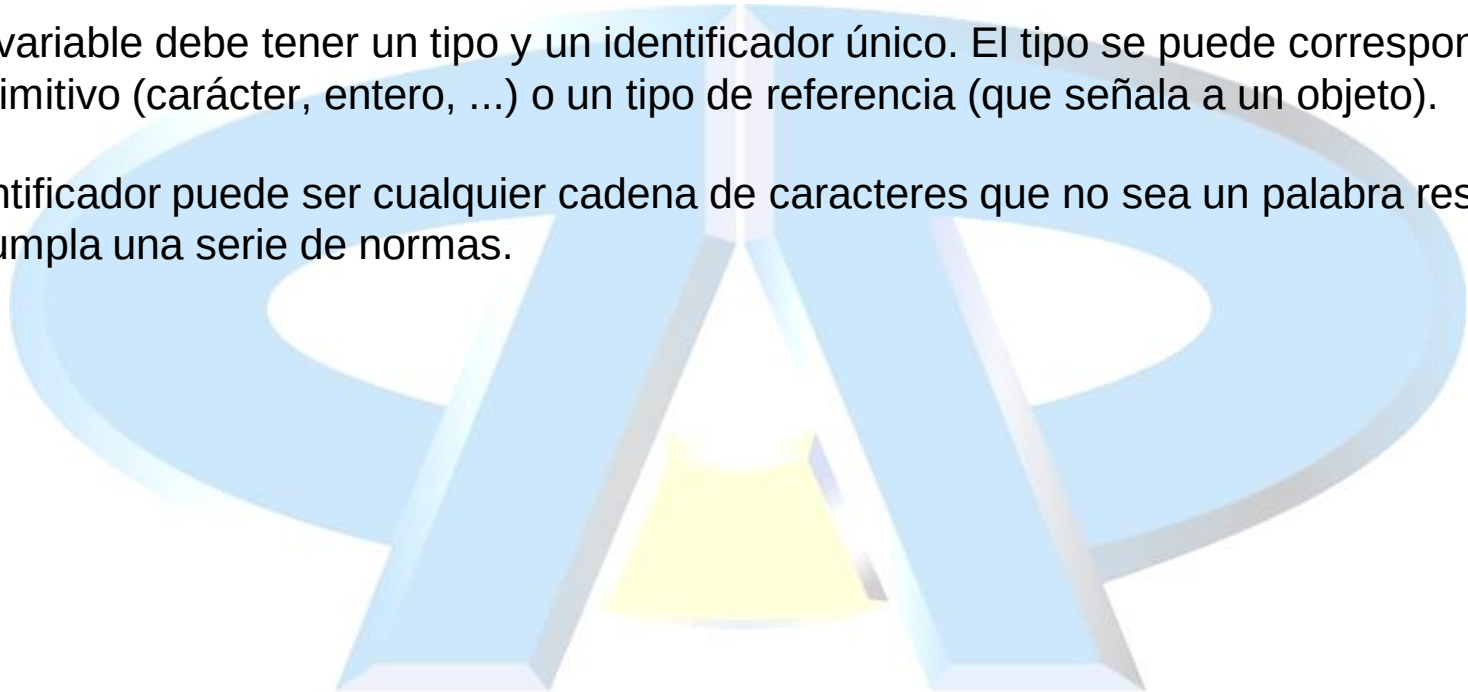




Identificadores, palabras clave y tipos

Variables

- Es el término Java para referirse a los atributos de una clase.
- Cada variable debe tener un tipo y un identificador único. El tipo se puede corresponder con tipo primitivo (carácter, entero, ...) o un tipo de referencia (que señala a un objeto).
- El identificador puede ser cualquier cadena de caracteres que no sea un palabra reservada y que cumpla una serie de normas.



Identificadores

- Los identificadores son nombres asignados a clases, variables y métodos.
- El **primer carácter** debe ser alguno de los siguientes:
 - Una letra mayúscula (A-Z).
 - Una letra minúscula (a-z).
 - El carácter subrayado (`_`)
 - El carácter (`$`)
- Los nombres de las clases **deben** (por convenio) comenzar por letra mayúscula. Los nombres de las variables y los métodos **deben** (por convenio) empezar por letra minúscula.
- El **segundo carácter** y siguientes pueden ser:
 - Cualquiera de los anteriores.
 - Caracteres numéricos (0-9).
- En identificadores de más de una palabra, se **debe** (por convenio) comenzar cada nueva palabra con mayúscula, sin usar separadores.
- No **pueden** usarse palabras clave de Java.
- El lenguaje de programación Java es sensible a mayúsculas y minúsculas.

Tipos primitivos y tipos de referencia

- Todas las variables tienen un tipo Java, primitivo o de referencia.
- Los tipos restringen los valores que pueden asignarse a una variable.
- La sintaxis de la declaración de una variable es:
 - ***tipo identificador_variable***
- **Tipos primitivos:** Son tipos de variables que permiten almacenar números enteros, decimales, valores lógicos, etc. Son los tipos de datos sencillos en Java.
- **Tipos de Referencia:** Se utilizan para almacenar la dirección de un objeto. Son punteros a objetos (Java si usa punteros, pero no utiliza aritmética de punteros).

Tipos primitivos

Tipos numéricos enteros:

- byte
- short
- int
- long

Tipos numéricos en coma flotante o reales:

- float
- double

Tipo texto: char

Tipo lógico: boolean



Tipos numéricos enteros.

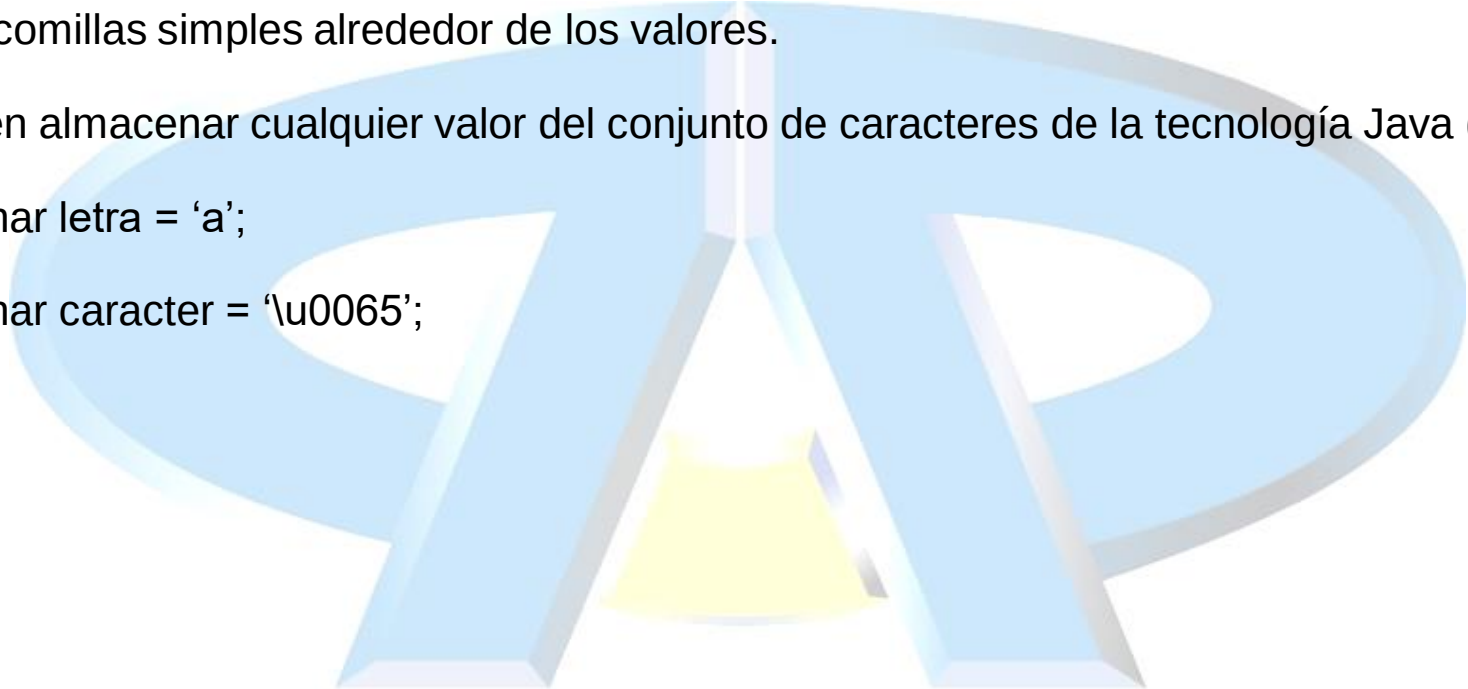
- Sirven para almacenar números enteros.
- Consideran el cero como un número positivo.
 - byte: 1 byte → -128 a 127
 - short: 2 bytes → -32.768 a 32.767
 - int: 4 bytes → -2.147.483.648 a 2.147.483.647
 - long: 8 bytes → -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807
- Almacenan números positivos y negativos.

Tipos numéricos en coma flotante

- Los tipos coma flotante almacenan números con partes decimales.
- No es posible definir el valor más grande y el más pequeño, porque este tipo de variables dependen de la precisión.
 - Representados según el estándar IEEE 754
- Existen dos constantes predefinidas, `POSITIVE_INFINITY` y `NEGATIVE_INFINITY`, para representar los límites de estos tipos.
- Tamaño en bytes:
 - float: 4 bytes
 - double: 8 bytes

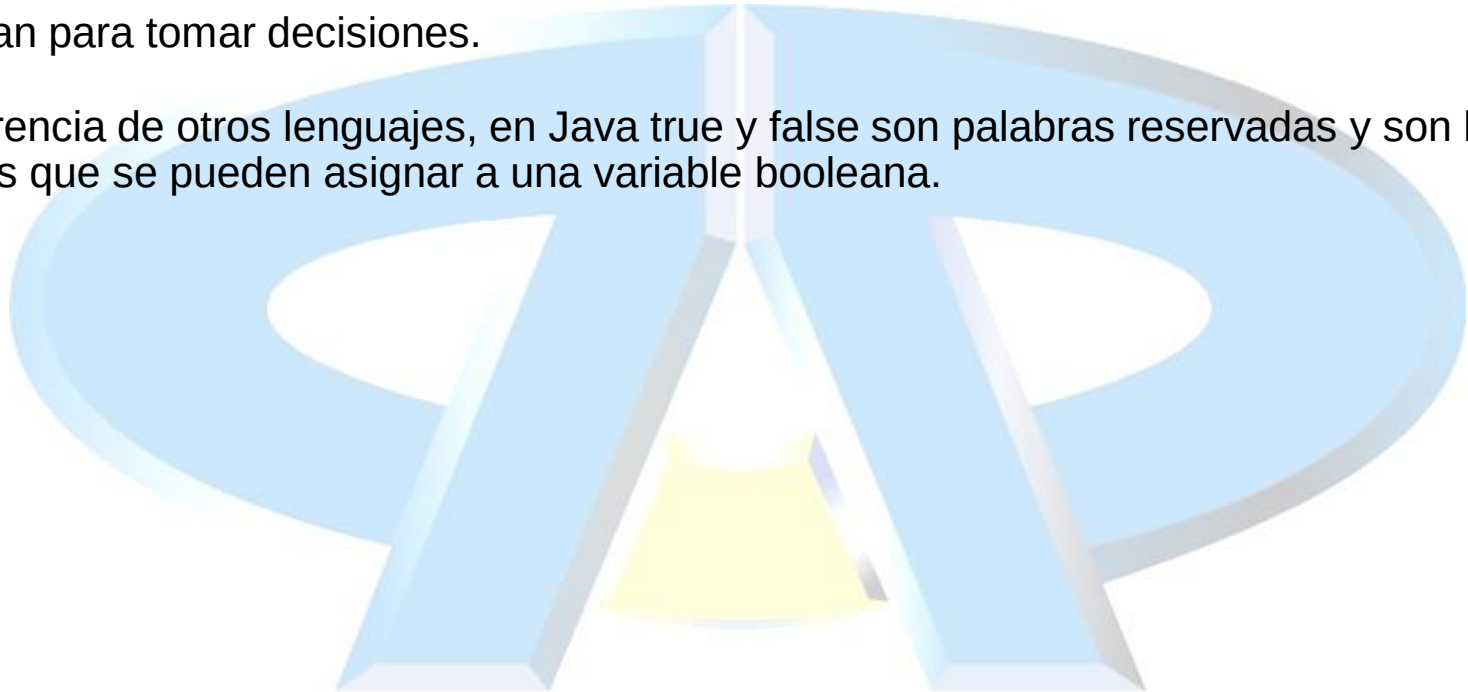
Tipo texto: char

- Almacenan un carácter.
- Usan comillas simples alrededor de los valores.
- Pueden almacenar cualquier valor del conjunto de caracteres de la tecnología Java (Unicode).
 - `char letra = 'a';`
 - `char caracter = '\u0065';`



Tipo lógico: boolean

- Almacenan uno de dos valores: true o false.
- Se usan para tomar decisiones.
- A diferencia de otros lenguajes, en Java true y false son palabras reservadas y son los únicos valores que se pueden asignar a una variable booleana.



Tipos de referencia

- Una vez vistos todos los tipos de datos primitivos, el resto de tipos que se encuentran en Java se refieren a objetos.
- Las variables que se refieren a objetos se denominan variables de referencia.

```
public class MiFecha {  
    private int day = 1;  
    private month = 1;  
    private year = 2003;  
    public MiFecha(int day, int month, int year) {...}  
    public void print() {...}  
}  
  
public class TestMiFecha {  
    public static void main(String args[]){  
        MiFecha hoy = new MiFecha(22, 7, 1975);  
    }  
}
```

Construcción e inicialización de objetos.

- Palabra clave ***new***:
 1. Primero, se reserva y se asigna el espacio para el nuevo objeto y se inicializa a 0 o null. En Java estas dos acciones son indivisibles para asegurar que los objetos no se inicializan con valores aleatorios
 2. Se realiza cualquier inicialización implícita
 3. Se ejecuta un constructor
 4. Se asigna a la variable de referencia el nuevo objeto creado en memoria

Asignación de memoria.

```
MiFecha hoy = new MiFecha(22, 7, 2003);
```

- MiFecha hoy → Reservar espacio solo para la referencia (puntero).
- new → Reserva e inicializa la memoria que se va a asignar al objeto.
- MiFecha(22, 7, 2003) → Llamada al constructor.
- = → Asignación a la variable de referencia del objeto.

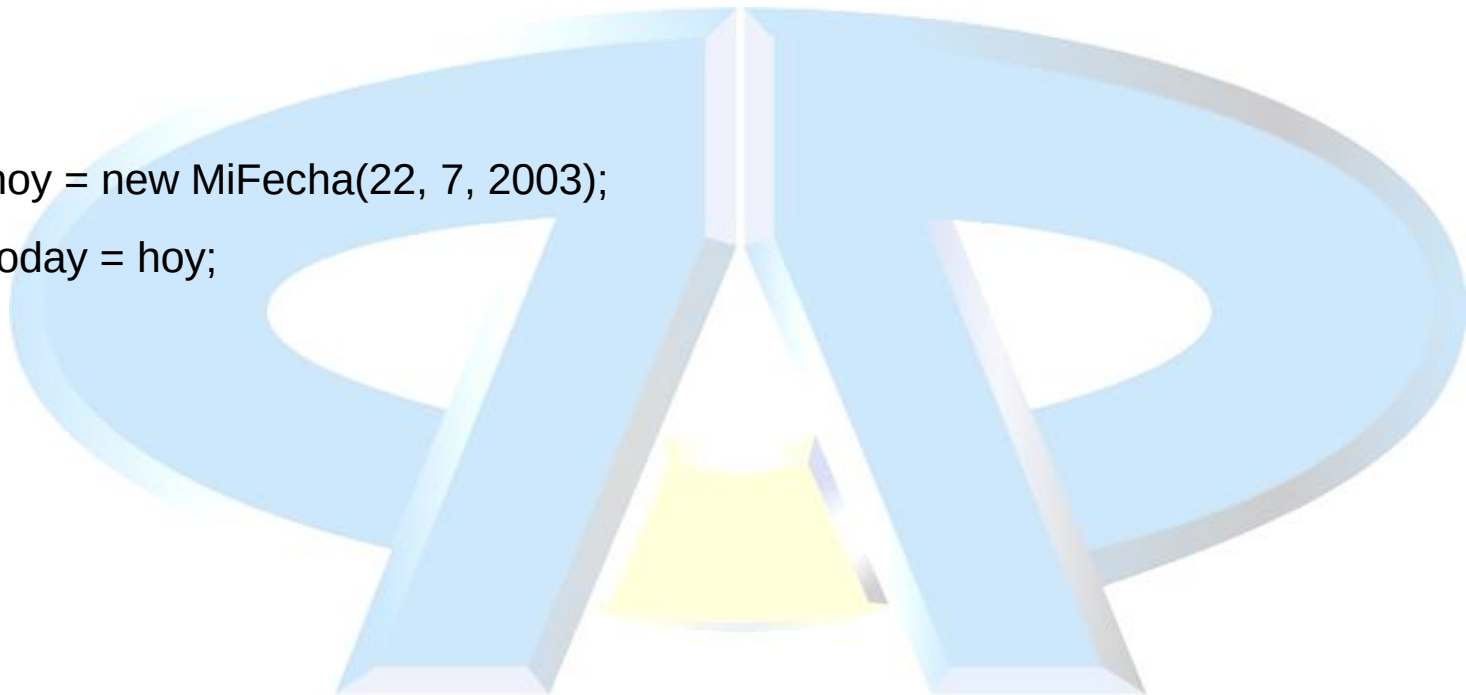
Asignación de variables por referencia.

```
int x = 7;
```

```
int y = x;
```

```
MiFecha hoy = new MiFecha(22, 7, 2003);
```

```
MiFecha today = hoy;
```



Paso de parámetros por valor.

- ***“El lenguaje de programación Java pasa los argumentos sólo por valor”***
 - El argumento no es cambiado por el método invocado.
 - Sin embargo, ¿qué sucede cuando se pasa como parámetro una variable de referencia?
 - ***Cuando se pasa como parámetro una variable de referencia, el contenido del “objeto” va a ser modificado por las acciones del método invocado, pero no el objeto en sí.***

Ejemplo

```
public class PassTest {  
    // Methods to change the current values  
    public static void changeInt(int value) {  
        value = 55;  
    }  
  
    public static void changeObjectRef(MyDate ref) {  
        ref = new MyDate(1, 1, 2000);  
    }  
  
    public static void changeObjectAttr(MyDate ref) {  
        ref.setDay(4);  
    }  
}
```

Ejemplo

```
public static void main(String args[]) {  
    MyDate date;  
    int val;                // Assign the int  
    val = 11;               // Try to change it  
    changeInt(val);         // What is the current value?  
  
    System.out.println("Int value is: " + val);  
  
    date = new MyDate(22, 7, 1964); // Assign the date  
    changeObjectRef(date);          // Try to change it  
    date.print();                   // What is the current value?  
    // Now change the day attribute through the object reference  
    changeObjectAttr(date);  
    // What is the current value?  
    date.print();  
}  
}
```


La referencia *this*

- Hacer referencia a atributos locales y métodos miembro, dentro de un método local o un constructor
- Pasar el objeto actual como argumento a otro método

